

ACRA

Augmented Cyber Risk Analysis

Rapport	Audit de sécurité OWASP WSTG
Version	3.0 — Révision post-implémentation
Date	07 June 2026
Auditeur	Analyse automatisée + revue de code
Périmètre	Application ACRA v3 — stack Next.js 14 / Prisma / PostgreSQL
Référentiel	OWASP Web Security Testing Guide 4.2
Classification	CONFIDENTIEL

Score de sécurité global

82 / 100

Niveau : BON (+7 pts vs V2)

Table des matières

1. Résumé exécutif	3
2. Périmètre et méthodologie	4
3. Nouvelles fonctionnalités auditées (V3)	5
4. Résultats détaillés par catégorie WSTG	6
4.1 Authentification (WSTG-ATHN)	6
4.2 Autorisation (WSTG-ATHZ)	7
4.3 Validation des entrées (WSTG-INPV)	8
4.4 Gestion des sessions (WSTG-SESS)	9
4.5 Cryptographie (WSTG-CRYP)	10
4.6 Gestion des erreurs (WSTG-ERRH)	10
4.7 Configuration et infrastructure (WSTG-CONF)	11
4.8 Logique métier (WSTG-BUSL)	11
4.9 Côté client (WSTG-CLNT)	12
5. Tableau récapitulatif des vulnérabilités	13
6. Matrice de risques	14
7. Recommandations priorisées	15
8. Comparaison V1 / V2 / V3	16
9. Conclusion	17
Annexe A Fichiers audités	18

1. Résumé exécutif

Ce rapport présente la troisième itération de l'audit de sécurité OWASP WSTG de l'application **ACRA (Augmented Cyber Risk Analysis)**, une application web guidant des non-spécialistes dans la réalisation d'analyses de risques EBIOS RM compatibles ISO 27005.

Depuis la version 2, plusieurs fonctionnalités significatives ont été ajoutées : gestion des entités organisationnelles (nouveau modèle *OrganizationConfig*), champ *entité* sur les mesures de sécurité, filtres avancés sur la page */actions* (statut, entité, échéance), export multi-format enrichi (CSV/Excel/PDF) et migration de jspdf vers *@react-pdf/render* côté serveur.

Catégorie WSTG	V1	V2	V3	Tendance
Authentification	60%	90%	92%	■
Autorisation (RBAC)	55%	88%	92%	■
Validation des entrées	50%	75%	78%	■
Gestion des sessions	65%	92%	92%	→
Cryptographie	70%	90%	90%	→
Gestion des erreurs	60%	85%	88%	■
Configuration	45%	88%	90%	■
Logique métier	60%	80%	82%	■
Côté client (XSS/CSRF)	50%	78%	80%	■
Score global	57%	75%	82%	■■

Le score global progresse de **75 % (V2) à 82 % (V3)**, soit +7 points. Cette amélioration reflète la mise en place du rate limiting multi-contexte, la migration des CVE bibliothèques (xlsx→exceljs), la génération PDF côté serveur, et la robustesse accrue des nouvelles API.

Points saillants V3

- **Nouveau** : API OrganizationConfig avec autorisation ADMIN stricte (PUT) et accès authentifié (GET)
- **Nouveau** : Rate limiting couvrant 7 contextes distincts (login, register, password, export, import, search, api-write)
- **Corrigé** : Remplacement de jspdf (client-side) par @react-pdf/render (serveur) — élimine l'exposition des données dans le navigateur
- **Corrigé** : Remplacement de xlsx 0.18 (CVE-2023-30533) par ExcelJS sans vulnérabilité connue
- **Résiduel** : Validation de longueur absente sur le champ entite (mesures) et les items de l'OrganizationConfig
- **Résiduel** : CSP script-src contient encore unsafe-inline (contrainte Next.js/CSS-in-JS)
- **Résiduel** : Rate limiting absent sur GET /api/admin/organization-config
- **Dette technique** : Cast (prisma as any) nécessaire à 3 endroits (champs post-migration sans regenerate local)

2. Périmètre et méthodologie

2.1 Périmètre

L'audit couvre l'intégralité de l'application ACRA dans sa version 3, incluant toutes les routes API, composants React, middlewares, configuration serveur et dépendances tierces.

Composant	Périmètre	Méthode
API Routes	/api/auth, /api/analyses, /api/workshop, /api/export, /api/import, /api/admin/*, /api/admin/organization-config	Revue statique
Middleware	src/middleware.ts (withAuth)	Revue statique
Bibliothèques Auth	next-auth v4.24.14, bcryptjs, JWT	Revue config + CVE
ORM / DB	Prisma 5.x + PostgreSQL 16	Revue schéma + requêtes
Headers HTTP	next.config.js (CSP, HSTS, X-Frame...)	Analyse config
Rate Limiting	rate-limit.ts (7 contextes)	Revue logique
Logging / Audit	logger.ts (Winston), /admin/audit	Revue complète
Nouvelles features V3	OrganizationConfig, entité sur Mesure, /actions filtres, exports enrichis	Focus V3

2.2 Référentiel appliqué

- OWASP Web Security Testing Guide (WSTG) version 4.2
- OWASP Top 10 2021
- CWE/SANS Top 25 Most Dangerous Software Errors
- RGPD — Règlement Général sur la Protection des Données
- RFC 6585 — Additional HTTP Status Codes (429 Too Many Requests)

2.3 Méthode de notation

Chaque vulnérabilité est évaluée selon une matrice **Impact × Probabilité** donnant une sévérité sur 4 niveaux. Le score global est calculé comme la moyenne pondérée des catégories WSTG, où chaque catégorie est notée de 0 à 100 % selon le taux de contrôles satisfaits.

Sévérité	CVSS approx.	Impact	Action recommandée
CRITIQUE	9.0 – 10.0	Compromission complète possible	Corriger immédiatement
ÉLEVÉ	7.0 – 8.9	Exposition de données sensibles	Corriger avant mise en prod
MOYEN	4.0 – 6.9	Risque limité, conditions d'exploitation	Planifier correction
FAIBLE	0.1 – 3.9	Risque très limité	Corriger au prochain sprint
INFO	0.0	Amélioration ou bonne pratique	À considérer

3. Nouvelles fonctionnalités auditées (V3)

Cette section présente spécifiquement les fonctionnalités introduites ou modifiées depuis la V2, avec leur évaluation de sécurité initiale.

Fonctionnalité	Fichiers impactés	Évaluation
OrganizationConfig (GET/PUT)	/api/admin/organization-config/route.ts	■ ADMIN-only sur PUT, auth sur GET, sanitisation OK
Champ entite sur Mesure	schema.prisma, workshop/[id] API case 5	■■■ Aucune validation longueur max côté serveur
Datalist entités dans Atelier5	Atelier5.tsx	■ Données lues depuis API auth, pas de XSS (React escaping)
Filtres /actions (statut/entité/échéance)	ActionsClient.tsx, actions/page.tsx	■ Filtrage client-side, aucune donnée supplémentaire exposée
Export enrichi (entite dans CSV/Excel/PDF)	export/[id]/route.ts, pdf-template.tsx	■ canViewAnalyse vérifié, rate limit 20/h
resultCount : string template (fix SSR)	i18n/fr.ts et 4 autres, ActionsClient.tsx	■ Correction d'une erreur de type Server→Client boundary
Migration.xlsx→ExcelJS	export/[id]/route.ts, package.json	■ Élimine CVE-2023-30533 (SSRF/prototype pollution)
Migration jspdf→@react-pdf/renderer	pdf-template.tsx, export/[id]/route.ts	■ PDF généré côté serveur, plus d'exposition navigateur
Fix params async Next.js 15/16	7 fichiers route.ts et 2 pages	■ Corrige comportement indéterminé sur await params

4. Résultats détaillés par catégorie WSTG

4.1 Authentification — WSTG-ATHN (Score : 92/100)

WSTG-ATHN-01	Mécanismes de transport des credentials	■ PASSÉ
--------------	---	---------

Les credentials transitent exclusivement via HTTPS en production. Cookies sessionToken configurés HttpOnly, Secure, SameSite=Lax. En développement, Secure est désactivé intentionnellement.

WSTG-ATHN-02	Politique de mots de passe	■ PASSÉ
--------------	----------------------------	---------

PasswordPolicy configurable par l'admin : longueur min, majuscules, chiffres, caractères spéciaux. Validation côté serveur dans /api/user/password et /api/auth/register.

WSTG-ATHN-03	Fonctionnalité de verrouillage de compte	■ PASSÉ
--------------	--	---------

Rate limiting de 10 tentatives / 15 min par email via rate-limit.ts. Audit log LOGIN_RATE_LIMITED généré. Compte isActive vérifié avant comparaison bcrypt.

WSTG-ATHN-04	Contournement du schéma d'authentification	■ PASSÉ
--------------	--	---------

Middleware withAuth protège toutes les routes sauf /auth/*, /, /api/auth/*, /legal. Vérification redondante dans chaque API route via getSessionServer.

WSTG-ATHN-05	Fonctionnalité de remember me	■ N/A
--------------	-------------------------------	-------

Pas de fonctionnalité "remember me". Durée de session fixée à 8h (JWT maxAge).

WSTG-ATHN-06	Failles dans le cache navigateur	■ PASSÉ
--------------	----------------------------------	---------

Headers Cache-Control: no-store sur toutes les routes /api. Pages sensibles marquées force-dynamic dans Next.js.

WSTG-ATHN-07	Politique de mot de passe faible	■ PASSÉ
--------------	----------------------------------	---------

Implémentation dans password-policy.ts, testée par TDD (Vitest). Règles communiquées au frontend avec affichage conditionnel.

WSTG-ATHN-08	Réinitialisation de mot de passe	■ N/A
--------------	----------------------------------	-------

Pas de fonctionnalité de réinitialisation par email implémentée. L'administrateur peut changer les mots de passe via l'interface d'administration.

4.2 Autorisation — WSTG-ATHZ (Score : 92/100)

WSTG-ATHZ-01	Répertoire de traversée des droits	■ PASSÉ
--------------	------------------------------------	---------

permissions.ts centralise la matrice RBAC avec 5 rôles. canViewAnalyse, canEditAnalyse, canApproveAnalyse vérifiés sur chaque API route concernée. Aucune route ne fait confiance aux paramètres côté client pour l'autorisation.

WSTG-ATHZ-02	Contournement du schéma d'autorisation	■ PASSÉ
--------------	--	---------

La nouvelle API OrganizationConfig vérifie explicitement userRole === 'ADMIN' sur PUT. GET est intentionnellement ouvert à tous les utilisateurs authentifiés (nécessaire pour peupler les datalists dans Atelier5).

WSTG-ATHZ-03	Escalade de privilèges	■ PASSÉ
--------------	------------------------	---------

Le rôle est lu depuis la DB à chaque rafraîchissement de token JWT (callback jwt), empêchant l'utilisation d'un ancien token avec un rôle révoqué. L'admin ne peut pas se suspendre lui-même.

WSTG-ATHZ-04	Référence directe aux objets (IDOR)	■ PASSÉ
--------------	-------------------------------------	---------

Toute requête sur une analyse vérifie l'appartenance ou l'accès partagé via analyseWhereClause(). L'export /api/export/[id] applique canViewAnalyse avant tout accès aux données.

4.3 Validation des entrées — WSTG-INPV (Score : 78/100)

La validation des entrées reste le domaine avec le plus de marge de progression. Les nouvelles fonctionnalités V3 introduisent des points d'amélioration identifiés.

WSTG-INPV-01	Réflexion XSS (Cross-Site Scripting)	■ PASSÉ
React échappe automatiquement les valeurs dans le JSX. Aucun usage de dangerouslySetInnerHTML détecté. Les données i18n utilisent des string templates (plus de fonctions). La CSP script-src inclut toujours unsafe-inline (contrainte Next.js/CSS-in-JS).		
WSTG-INPV-02	Injection SQL	■ PASSÉ
Prisma ORM avec requêtes paramétrisées sur 100 % des accès DB. Aucune requête brute (prisma.\$queryRaw) détectée dans le code audité.		
WSTG-INPV-03	Injection HTTP	■ PASSÉ
Content-Disposition dans les routes export utilise un sanitize regex [^a-zA-Z0-9\-_] avant insertion dans le header, limitant les injections CRLF.		
WSTG-INPV-05	Validation longueur — OrganizationConfig	■■ MOYEN
Le PUT /api/admin/organization-config n'impose pas de limite sur la longueur des noms d'entités (String(e).trim()) ni sur le nombre d'entités dans le tableau. Un admin malveillant pourrait stocker des valeurs excessivement longues en base. Recommandation : limiter à 50 caractères par entité et 30 entités max.		
WSTG-INPV-06	Validation longueur — champ entite sur Mesure	■■ MOYEN
Le champ entite (String? nullable) dans le modèle Mesure n'a pas de contrainte de longueur explicite dans schema.prisma. Il est passé via ...rest dans le workshop API, sans validation Zod côté serveur. Recommandation : ajouter @db.VarChar(100) dans Prisma et une validation Zod dans le handler workshop case 5.		
WSTG-INPV-10	Mass assignment — import JSON	■■ MOYEN
La fonction clean() dans /api/import/route.ts supprime les champs DB (id, analysed, createdAt...) mais passe le reste via ...rest à Prisma create. Les nouveaux champs (entite, etc.) ajoutés au schéma après la V2 sont acceptés depuis un fichier importé sans validation explicite. Recommandation : utiliser une allowlist stricte des champs acceptés à l'import.		
WSTG-INPV-11	Limite taille payload import	■ PASSÉ
Le handler POST /api/import vérifie explicitement body.data.length > 2 * 1024 * 1024 et retourne HTTP 413 si dépassé.		

4.4 Gestion des sessions — WSTG-SESS (Score : 92/100)

La gestion des sessions n'a pas évolué depuis la V2. Le niveau de sécurité reste élevé, maintenu par la configuration NextAuth et la vérification périodique de l'état du compte.

- ■ JWT avec maxAge 8h (vs 30j par défaut) — exposition réduite en cas de vol de token
- ■ Cookie HttpOnly, Secure (production), SameSite=Lax — protection CSRF et vol via XSS
- ■ Vérification isActive à chaque rafraîchissement JWT — suspension effective immédiatement
- ■ Aucun token dans les URL, logs ou localStorage
- ■■ Absence de révocation de session individuelle (logout ne détruit que le cookie local)
- ■■ NextAuth v4.24.14 épinglé — migration vers Auth.js v5 planifiée (tâche #189)

4.5 Cryptographie — WSTG-CRYP (Score : 90/100)

- ■ bcryptjs avec salt rounds par défaut (10) pour les mots de passe
- ■ NEXTAUTH_SECRET enforced au démarrage (check-env.ts) — erreur fatale si absent
- ■ HSTS max-age=31536000 includeSubDomains preload — activé en production uniquement
- ■ upgrade-insecure-requests dans la CSP — force le passage HTTPS pour les sous-ressources
- ■■ Pas de rotation automatique des clés NEXTAUTH_SECRET — recommandé annuellement
- ■■ PostgreSQL : chiffrement au repos non configuré dans docker-compose (hors périmètre dépoyeur)

4.6 Gestion des erreurs — WSTG-ERRH (Score : 88/100)

- ■ Aucun stack trace exposé aux réponses API — erreurs transformées en messages génériques
- ■ Logger Winston structuré — JSON en production, lisible en dev, compatible SIEM
- ■ Audit trail sur login (succès/échec/rate limit), accès admin, exports
- ■ Page admin /admin/audit pour consultation des logs — accès ADMIN uniquement
- ■■ Organisation config : aucun audit log sur PUT (modification de la liste des entités)
- ■■ Logs en /tmp/logs en production — persistance non garantie entre redémarrages Docker
- ■■ Recommandation : ajouter auditLog sur PUT /api/admin/organization-config

4.7 Configuration et infrastructure — WSTG-CONF (Score : 90/100)

- En-têtes de sécurité complets : X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy, Permissions-Policy
- CSP complète : default-src self, object-src none, frame-ancestors none, form-action self
- Headers anti-cache sur /api/* : Cache-Control no-store, must-revalidate
- Rate limiting sur 7 contextes (login, register, password, export, import, search, api-write)
- Docker multi-stage build, utilisateur non-root nextjs dans le conteneur applicatif
- Migrator Prisma séparé — élimine la race condition schéma/application au démarrage
- unsafe-inline dans script-src — requis par Next.js CSS-in-JS, affaiblit XSS protection
- Rate limiting absent sur GET /api/admin/organization-config (accessible à tous les users auth)
- In-memory rate limiter : ne résiste pas à un déploiement multi-instance (Redis recommandé en prod)
- Index Prisma ajoutés sur FK — performances protégées contre les scans complets

4.8 Logique métier — WSTG-BUSL (Score : 82/100)

- Workflow approbation EBIOS RM : statuts EN_COURS → SOUMIS → APPROUVE/REJETE/TERMINE respectés
- Un analyste ne peut pas approuver sa propre analyse
- Les transactions Prisma protègent l'intégrité des sauvegardes multi-tables (workshop API)
- La page /actions filtre côté client uniquement sur les données déjà autorisées côté serveur
- Le champ entite est un datalist libre : un utilisateur peut entrer une valeur non listée. Cohérence fonctionnelle à surveiller.
- Analyse express (ateliers 1+2+5) : les ateliers 3 et 4 peuvent contenir des données orphelines non liées
- Pas de limite sur le nombre de mesures par analyse — risque de données volumineuses à exporter

4.9 Côté client — WSTG-CLNT (Score : 80/100)

- Aucun usage de dangerouslySetInnerHTML dans les composants audités
- ThemeProvider stocke la préférence de thème dans localStorage uniquement (pas de données sensibles)
- Aucune transmission de données sensibles via window.postMessage ou BroadcastChannel
- frame-ancestors none dans la CSP — prévient le clickjacking et l'injection via iframes
- Correction V3 : resultCount passé comme string template (plus de fonction via boundary SSR→Client)
- unsafe-inline dans script-src affaiblit la défense XSS côté client — voir WSTG-CONF
- Recherche globale dans la Navbar : données filtrées côté client après un fetch de toutes les analyses accessibles. Pour de larges datasets, ce pattern pourrait exposer plus de données que nécessaire.

5. Tableau récapitulatif des vulnérabilités

ID	Titre	Sévérité	WSTG	Statut	Vers ion
V3-01	Absence de validation longueur sur OrganizationConfig entites	MOYEN	INPV-05	NOUVEAU	V3
V3-02	Absence de validation longueur sur Mesure.entite	MOYEN	INPV-06	NOUVEAU	V3
V3-03	Mass assignment via import JSON (nouveaux champs schéma)	MOYEN	INPV-10	RÉSIDUEL	V2+
V3-04	Rate limiting absent sur GET /api/admin/organization-config	FAIBLE	CONF	NOUVEAU	V3
V3-05	Absence d'audit log sur PUT /api/admin/organization-config	FAIBLE	ERRH	NOUVEAU	V3
V3-06	unsafe-inline dans CSP script-src	FAIBLE	CONF	RÉSIDUEL	V1+
V3-07	Rate limiter in-memory (non multi-instance)	FAIBLE	CONF	RÉSIDUEL	V2+
V3-08	Absence de rotation des clés NEXTAUTH_SECRET	FAIBLE	CRYP	RÉSIDUEL	V1+
V3-09	Logs /tmp/logs non persistants entre redémarrages	FAIBLE	ERRH	RÉSIDUEL	V2+
V3-10	Révocation de session individuelle absente	INFO	SESS	RÉSIDUEL	V1+
V2-01	Injection SQL (Prisma ORM)	CRITIQUE	INPV-02	■ CORRIGÉ V1	V1
V2-02	CVE-2023-30533 xlsx prototype pollution	ÉLEVÉ	CONF	■ CORRIGÉ V3	V2
V2-03	PDF généré côté client (exposition données)	ÉLEVÉ	CONF	■ CORRIGÉ V3	V2
V2-04	Session JWT 30j par défaut	MOYEN	SESS	■ CORRIGÉ V2	V1
V2-05	Absence de rate limiting	MOYEN	ATHN	■ CORRIGÉ V2	V1

Les identifiants V3-xx désignent les vulnérabilités actives. Les V2-xx sont listées pour traçabilité et indiquent leur statut de correction.

6. Matrice de risques

La matrice ci-dessous positionne les vulnérabilités actives selon leur impact potentiel et leur probabilité d'exploitation. Les vulnérabilités corrigées ne sont pas représentées.

↑ Impact	FAIBLE	MOYEN	ÉLEVÉ	CRITIQUE
ÉLEVÉE	V3-07 V3-10	—	—	—
MOYENNE	V3-06 V3-08 V3-09	V3-01 V3-02 V3-03	—	—
FAIBLE	V3-04 V3-05	—	—	—

VERT : Faible

JAUNE : Moyen

ORANGE : Élevé

ROUGE : Critique

7. Recommandations priorisées

■ Priorité 1 — À corriger avant déploiement

[V3-01/02] Valider la longueur des champs entite

Ajouter `@db.VarChar(100)` dans `schema.prisma` sur `Mesure.entite`. Dans `PUT /api/admin/organization-config`, limiter chaque entité à 50 caractères et le tableau à 30 items. Code : `.filter(e => e.length <= 50).slice(0, 30)`

[V3-03] Allowlist stricte à l'import JSON

Remplacer le `...rest passthrough` dans `importJSON()` par une liste blanche explicite des champs acceptés pour chaque modèle. Cela évite l'injection de champs non prévus si le schéma Prisma évolue.

■ Priorité 2 — Planifier au prochain sprint

[V3-04] Rate limiting sur GET /api/admin/organization-config

Ajouter `rateLimit(`org-config:${userId}`, LIMIT_SEARCH.limit, LIMIT_SEARCH.windowMs)` dans le handler GET. Cet endpoint est appelé à chaque ouverture d'Atelier5.

[V3-05] Audit log sur PUT /api/admin/organization-config

Appeler `auditLog('ORGANIZATION_CONFIG_UPDATED', {...})` après le `upsert` Prisma. Ajouter `ORGANIZATION_CONFIG_UPDATED` à l'enum `AuditAction` dans `schema.prisma`.

[V3-06] Durcir la CSP : remplacer unsafe-inline par des nonces

Implémenter un middleware Next.js générant un nonce CSP par requête. Remplacer `script-src 'unsafe-inline'` par `script-src 'nonce-{nonce}'`. Nécessite la compatibilité avec Tailwind CSS (peu d'impact en pratique).

■ Priorité 3 — Backlog technique

[V3-07] Migrer le rate limiter vers Redis

En production multi-instance (Kubernetes, load balancer), le store in-memory est insuffisant. Implémenter `RedisRateLimitStore` tel que documenté dans `rate-limit.ts`, activer via variable d'environnement `REDIS_URL`.

[V3-08] Rotation annuelle de NEXTAUTH_SECRET

Documenter une procédure de rotation : générer un nouveau secret (`openssl rand -base64 64`), déployer en rolling update. Les tokens existants seront invalidés — les utilisateurs devront se reconnecter.

[V3-09] Persister les logs audit

Monter un volume Docker persistant sur `/tmp/logs` ou utiliser un système de log centralisé (Elasticsearch, Loki, CloudWatch). Les logs actuels sont perdus au redémarrage du conteneur.

[V3-10] Implémenter la révocation de session individuelle

Maintenir une liste noire de JTI (JWT ID) en base ou Redis. Permettre à l'admin de révoquer toutes les sessions d'un utilisateur. Pertinent pour les scénarios de compromission de compte.

8. Comparaison V1 / V2 / V3

Ce tableau retrace l'évolution du niveau de sécurité de l'application ACRA depuis sa première version jusqu'à la version actuelle V3.

Domaine	V1 (initial)	V2 (sécurisation)	V3 (courant)
Authentification	JWT 30j, pas de RL	JWT 8h, RL 10/15min, bcrypt	Idem + isActive check continu
Autorisation	Pas de RBAC formel	RBAC 5 rôles, permissions.ts	RBAC étendu, API org-config ADMIN-only
Validation entrées	Aucune	Prisma ORM, 2MB limit import	Idem + sanitize filename export (partiel)
Gestion sessions	Défauts NextAuth	JWT sécurisé, cookies hardened	Idem (stable)
Cryptographie	bcrypt seul	+ HSTS, NEXTAUTH_SECRET enforced	Idem (stable)
Logging / Audit	Console.log	Winston structuré, audit trail	Idem + page admin audit
Headers HTTP	Aucun	CSP, X-Frame, HSTS...	Idem + optimisations mineures
Rate limiting	Aucun	7 contextes in-memory	Idem (gaps mineurs comblés partiellement)
Dépendances	xlsx CVE, jspdf client	xlsx → exceljs (V3)	ExcelJS + @react-pdf/renderer côté serveur
Score global	57 %	75 %	82 %

9. Conclusion

L'application ACRA v3 atteint un niveau de sécurité **bon (82/100)** selon le référentiel OWASP WSTG 4.2. Les fondations de sécurité posées en V2 (RBAC, rate limiting, sessions sécurisées, headers HTTP, audit logging) sont bien maintenues et renforcées dans cette version.

Les nouvelles fonctionnalités V3 — OrganizationConfig, champ entite, filtres /actions, exports enrichis — ont été développées avec une attention correcte à la sécurité. Deux vulnérabilités de sévérité **MOYENNE** ont été identifiées (absence de validation de longueur sur entite et OrganizationConfig), ainsi que des faiblesses de niveau **FAIBLE** héritées des versions précédentes.

Actions recommandées	Nombre	Effort estimé
Priorité 1 — Avant déploiement production	2	2-4h dev + test
Priorité 2 — Prochain sprint	3	4-8h dev + test
Priorité 3 — Backlog	4	2-5j selon complexité
Total	9	~2-3 sprints pour tout couvrir

En appliquant les recommandations Priorité 1, ACRA sera prête pour un déploiement en production. La mise en oeuvre des recommandations Priorité 2 (notamment la migration CSP vers les nonces et l'audit log OrganizationConfig) permettrait d'atteindre un score estimé à **88-90/100**, niveau **très bon** selon la grille WSTG.

La prochaine itération d'audit (V4) devrait se concentrer sur : la migration Auth.js v5 (#189), le typage Zod des champs JSON Prisma (#188), et les tests de pénétration dynamiques (DAST) en complément de la revue statique réalisée ici.

Annexe A — Fichiers audités

Fichier	Description
src/middleware.ts	Middleware authentication withAuth
src/lib/auth.ts	Configuration NextAuth — sessions, callbacks, JWT
src/lib/permissions.ts	Matrice RBAC centralisée
src/lib/rate-limit.ts	Rate limiter multi-contexte Redis-ready
src/lib/logger.ts	Logger Winston structuré + audit trail
src/lib/prisma.ts	Singleton Prisma client
src/lib/pdf-template.tsx	Template PDF @react-pdf/renderer côté serveur
src/app/api/admin/organization-config/route.ts	Nouveau — Config organisationnelle (GET/PUT)
src/app/api/admin/users/route.ts	Gestion utilisateurs ADMIN
src/app/api/export/[id]/route.ts	Export PDF/Excel/CSV — enrichi entite
src/app/api/import/route.ts	Import JSON/CSV avec rate limit + taille
src/app/actions/page.tsx	Nouveau — Page plan d'actions enrichie
src/components/ActionsClient.tsx	Nouveau — Filtres statut/entité/échéance
src/components/workshops/Atelier5.tsx	Atelier 5 — champ entite, datalist, masquage conditionnel
src/app/configuration/page.tsx	Config admin — section gestion entités
src/lib/i18n/fr.ts, en.ts, de.ts, es.ts, it.ts	Fichiers i18n — clés entite et filtres /actions
next.config.js	Headers sécurité, CSP, HSTS
docker-compose.yml	Configuration Docker — réseau, volumes, health check
prisma/schema.prisma	Modèle Mesure.entite, OrganizationConfig